Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
  - segment structure
  - reliable data transfer
  - flow control
  - connection management
- Principles of congestion control
- TCP congestion control

TCP: overview RFCs: 793,1122, 2018, 5681, 7323

- point-to-point:
  - one sender, one receiver
  - one to many communication not allowed
  - TCP Implemented in sender and receiver not in network core devices.
- reliable, in-order *byte steam:*
- ***Example:*** *If you send: Hello World*
- *TCP sees it like:* ***H e l l o W o r l d*** *→ a continuous flow of bytes.*
  - no “message boundaries"
- full duplex data:
  - bi-directional data flow in same connection
  - MSS: maximum segment size
  - MSS = Maximum amount of application data that TCP can send in one segment. It does NOT include:
  - TCP header
  - IP header
  - Only pure data/payload.

TCP: overview (No message boundaries)

TCP does not keep the original message structure.

Example: If the sender sends two messages:

- Message 1: "Hello“
- Message 2: "World“
- The receiver might get :

```
HelloW
orld
```

- Or

```
Hell
oWorld
```

Or both messages combined into one big chunk.

- TCP does NOT tell where message 1 ends. where message 2 begins Applications must create their own boundaries if they need them.

Cumulative Acknowledgements

Suppose the sender sends these segments:

- Segment 1 → bytes 1–100
- Segment 2 → bytes 101–200
- Segment 3 → bytes 201–300

Receiver gets Segment 1 and Segment 2.

The receiver sends:

👉 **ACK = 201**

This means:

- "I have received all bytes **up to 200**"
- "Send me data starting from byte 201"

Even though the receiver got two segments, it sends **one ACK**.

TCP: overview RFCs: 793,1122, 2018, 5681, 7323

- cumulative ACKs

pipelining: means sending multiple packets (segments) before waiting for ACKs. TCP does not send one segment and wait — instead it sends many segments continuously.

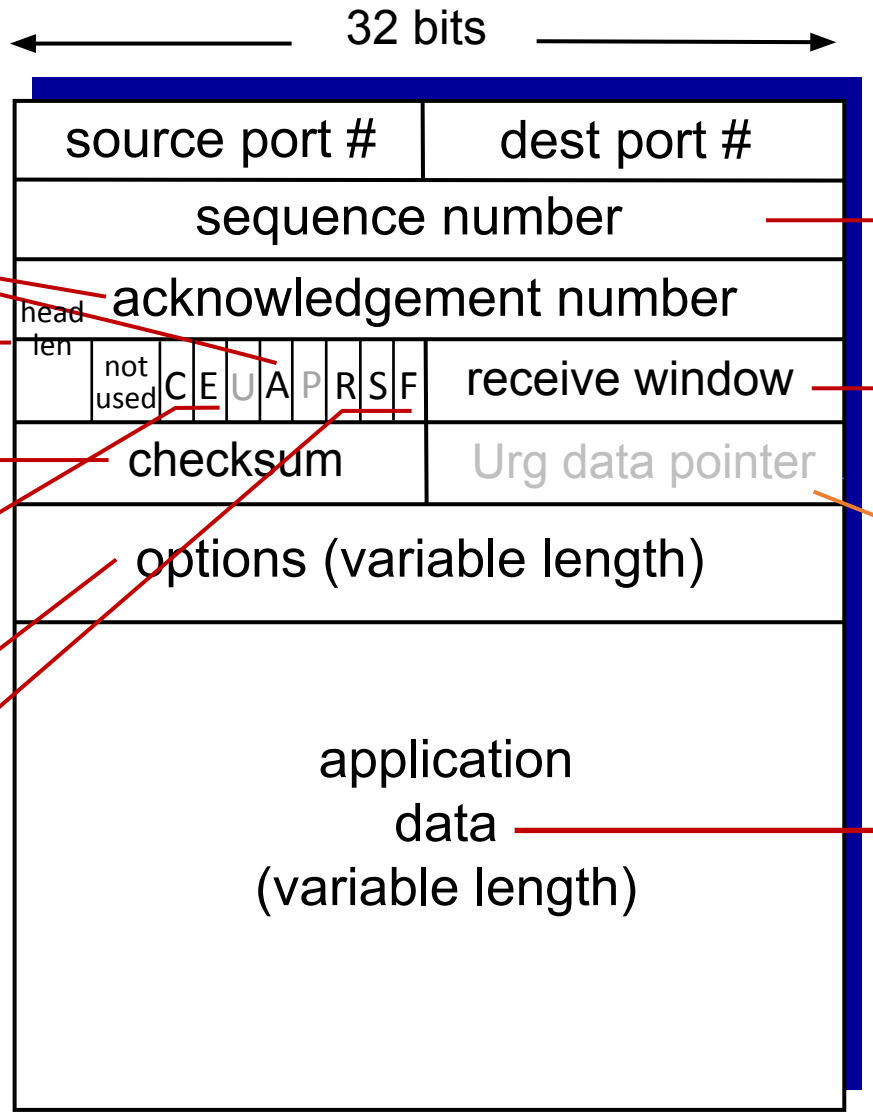
- Because TCP does pipelining (multiple segments at once).But it must not overload:
- the **receiver's buffer → flow control**
- the **network → congestion control**. So TCP uses a window to control sending speed.

connection-oriented:

- handshaking (exchange of control messages) initializes sender, receiver state before data exchange

- flow controlled:
  - sender will not overwhelm receiver

TCP segment structure



ACK: seq # of next expected byte; A bit: this is an ACK

length (of TCP header): Usually between 20–60 bytes

Internet checksum

C, E: congestion notification

TCP options: Used for: Maximum Segment Size (MSS), Window scaling

RST, SYN, FIN: connection management

segment seq #: counting bytes of data into bytestream (not segments!)

flow control: # bytes receiver willing to accept

Used when URG flag = 1
Points to urgent data location

data sent by application into TCP socket

Flags (Control Bits)

Flag	Meaning
SYN	start connection
ACK	acknowledgment
FIN	end connection
RST	reset connection
PSH	push data immediately
URG	urgent data

TCP sequence numbers, ACKs

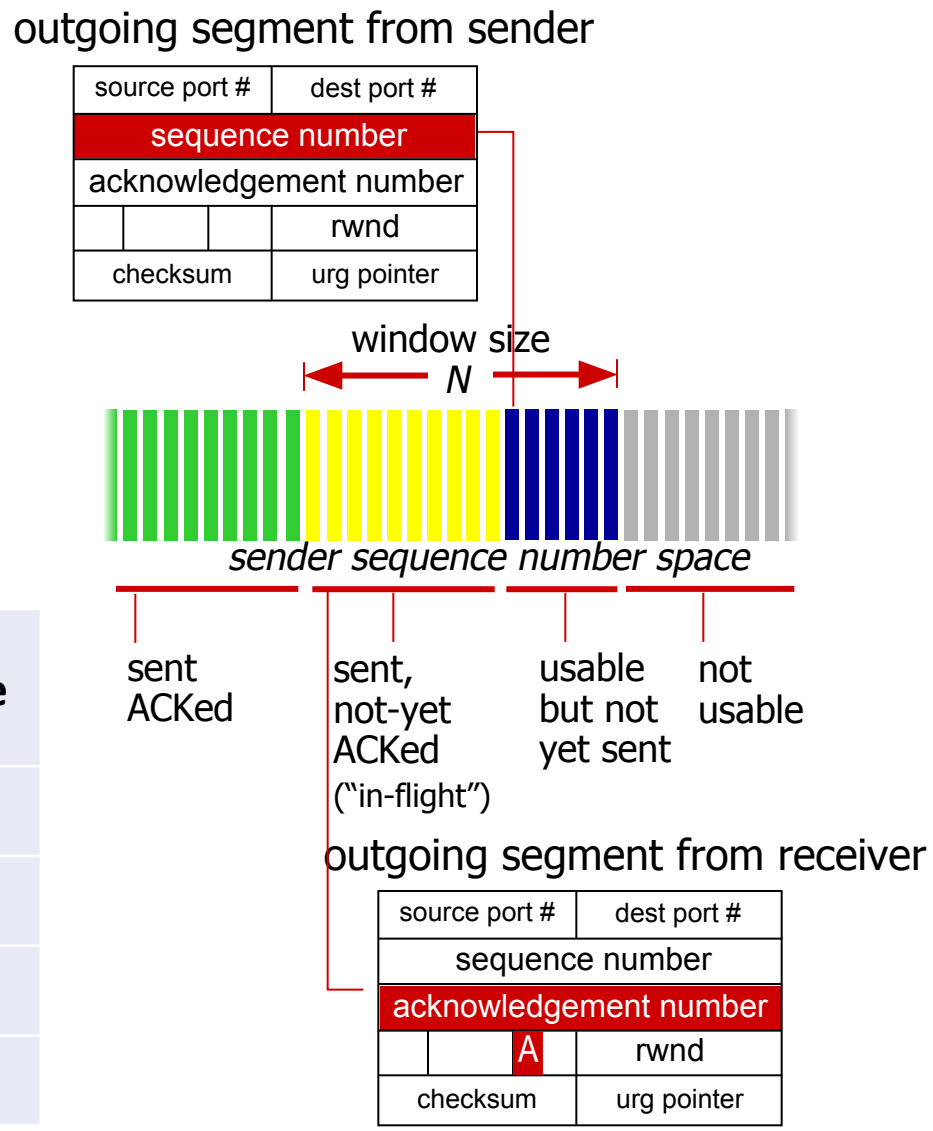
Sequence numbers:

- byte stream "number" of first byte in segment's data

Example:

- Assume MSS =1000 Bytes.
- Byte numbers:
- 0 – 999
- 1000 – 1999
- 2000 – 2999
- 3000 – 3999

Segment	Sequence Number	Bytes Inside
1	0	0–999
2	1000	1000–1999
3	2000	2000–2999
4	3000	3000–3999



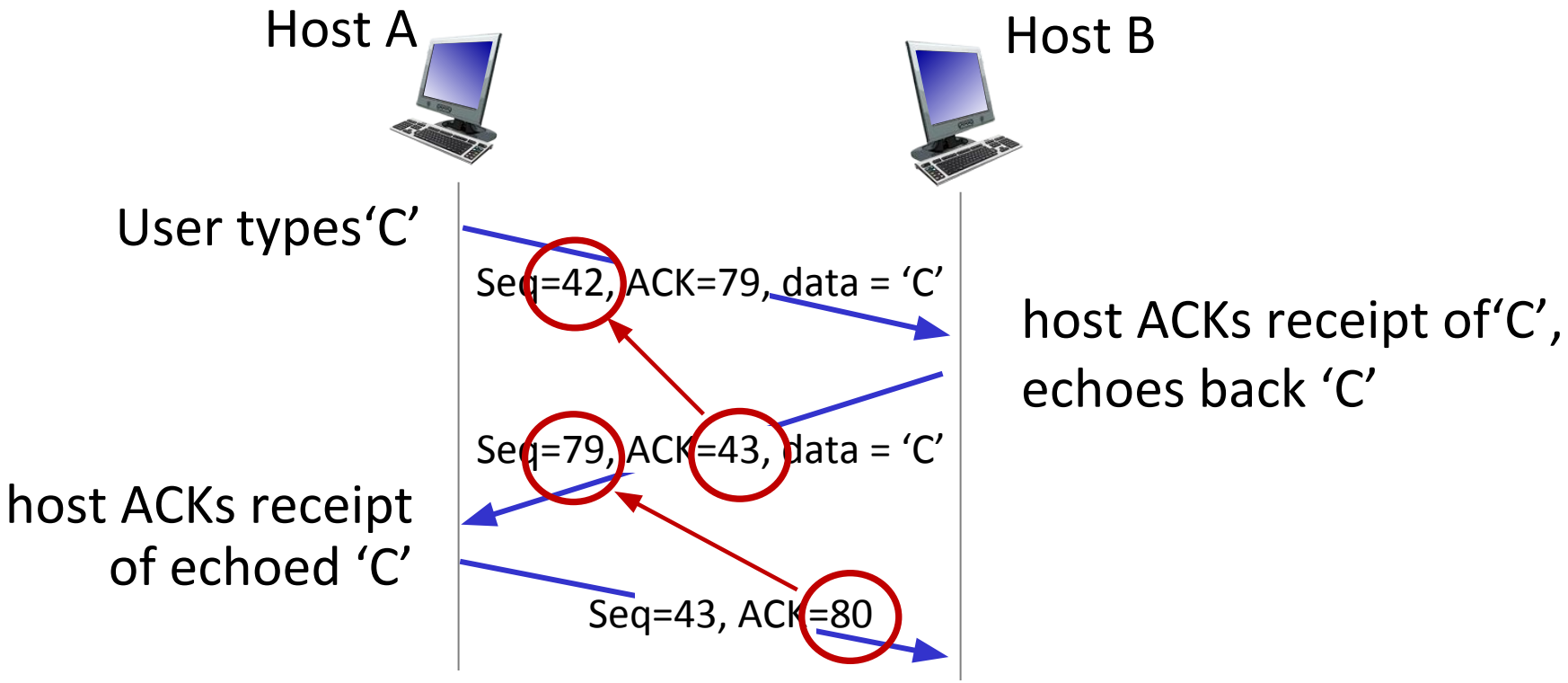
Acknowledgements:

- **Acknowledgment number = the next byte the receiver expects to receive.**
- It tells the sender:
- "I have successfully received all bytes **up to (ACK–1)**."
  - cumulative ACK

Q: how receiver handles out-of-order segments

- *A:* Receiver handling of out-of-order TCP segments is not fixed. TCP only requires final delivery to be in order, but the internal handling can vary by system.

TCP sequence numbers, ACKs



simple telnet scenario

TCP round trip time, timeout

Q: how to set TCP timeout value?

- TCP must choose a good timeout value for retransmissions. But RTT (Round-Trip Time) is not constant — it changes based on network traffic. So we must set timeout carefully.
- longer than RTT, but RTT varies!
- *too short: TCP will assume packet is lost even if it is just delayed.*
- premature timeout, unnecessary retransmissions
- *too long:* TCP waits too long before retransmitting a truly lost packet.
- Slow recovery
- Higher delay
- Bad performance

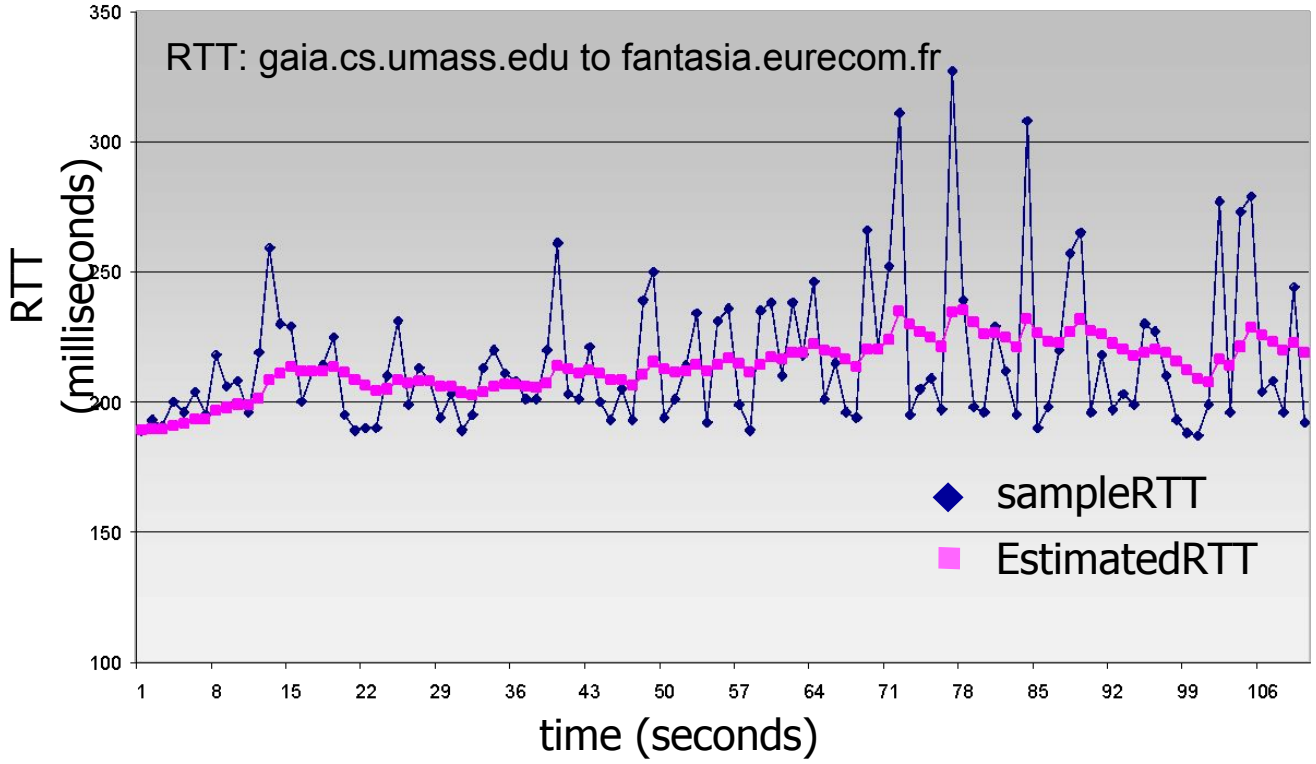
Q: how to estimate RTT?

- `SampleRTT:` measured time from segment transmission until ACK receipt
  - ignore retransmissions
- `SampleRTT` will vary, means every time you send a segment and get an ACK, the measured round-trip time (SampleRTT) might be different."
  - These fluctuations happen because of network congestion, routing changes, etc

TCP round trip time, timeout

$$\text{EstimatedRTT} = (1-\alpha)*\text{EstimatedRTT} + \alpha*\text{SampleRTT}$$

- exponential weighted moving average (EWMA)
- influence of past sample decreases exponentially fast
- typical value: $\alpha = 0.125$



TCP round trip time, timeout

- timeout interval: **EstimatedRTT** plus "safety margin"
  - large variation in **EstimatedRTT**: want a larger safety margin

```
TimeoutInterval = EstimatedRTT + 4*DevRTT
```

estimated RTT

"safety margin"

- **DevRTT**: EWMA of **SampleRTT** deviation from **EstimatedRTT**:

```
DevRTT = (1-β)*DevRTT + β*|SampleRTT-EstimatedRTT|
```

(typically, β = 0.25)

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

TCP Sender (simplified)

event: data received from application Means When the application gives TCP some data to send:

- Tcp create segment with seq #
- seq # is byte-stream number of first data byte in segment
- TCP uses a **retransmission timer** to handle lost segments.
- start timer if not already running
  - (because previous segments haven't been ACKed yet), don't restart it.
  - expiration interval: If the timer expires, it means the oldest unacknowledged segment may be lost. TCP will then retransmit that segment. The interval for this timer is called TimeOutInterval.

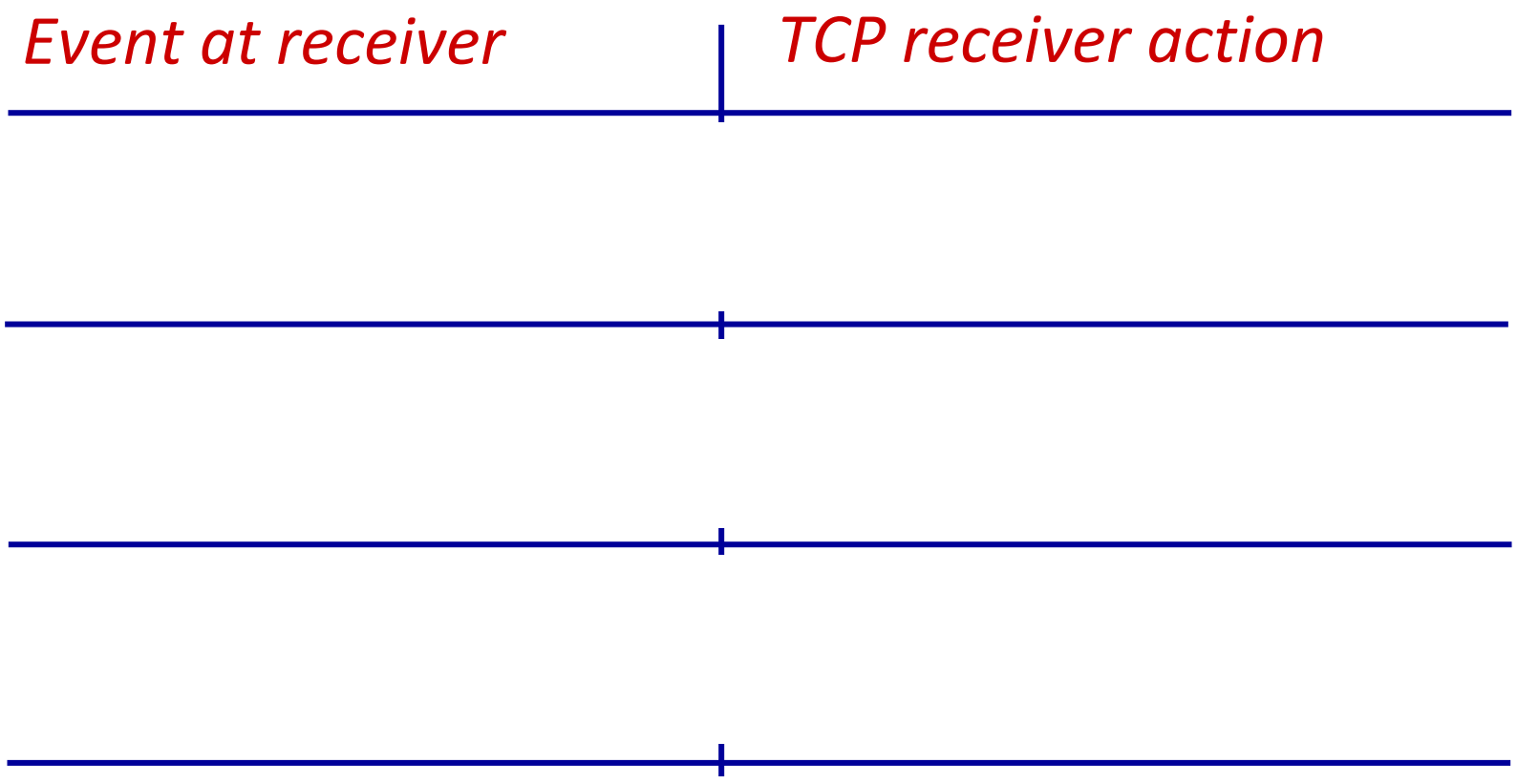
event: timeout

- retransmit segment that caused timeout
- restart timer

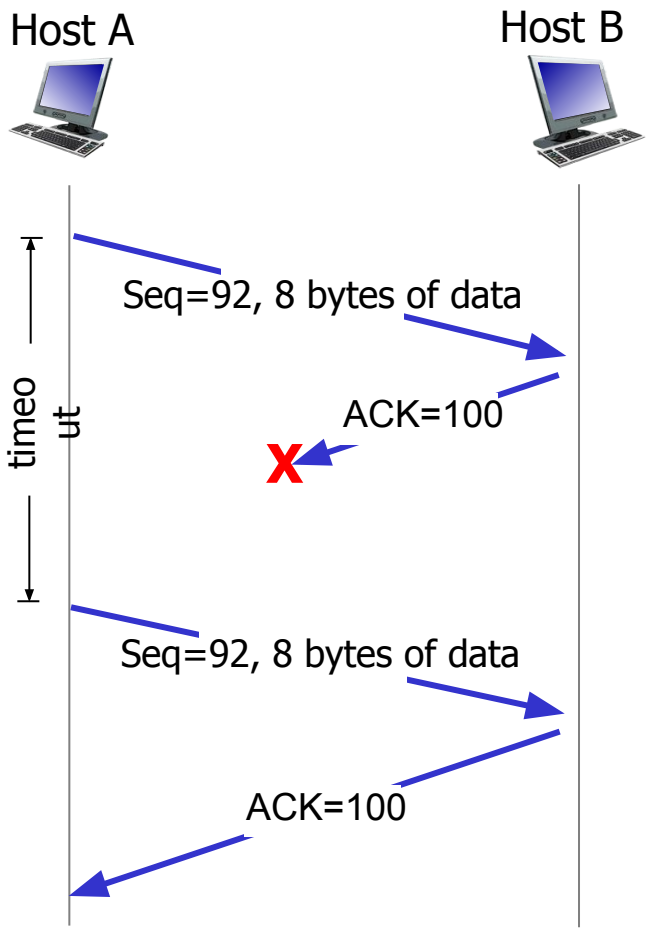
event: ACK received

- if ACK acknowledges previously unACKed segments
  - Update your record of what bytes have been successfully received by the receiver.
  - start timer if there are still unACKed segments

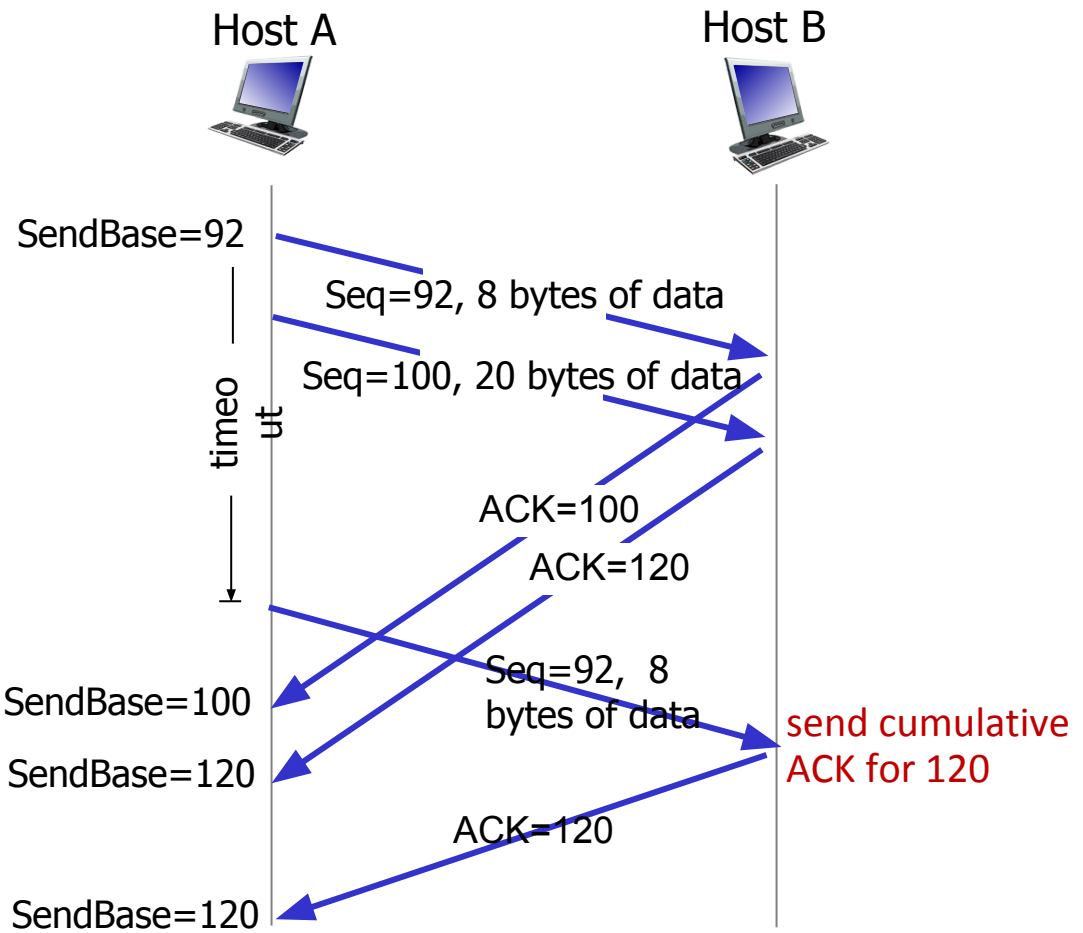
TCP Receiver: ACK generation [RFC 5681]



TCP: retransmission scenarios

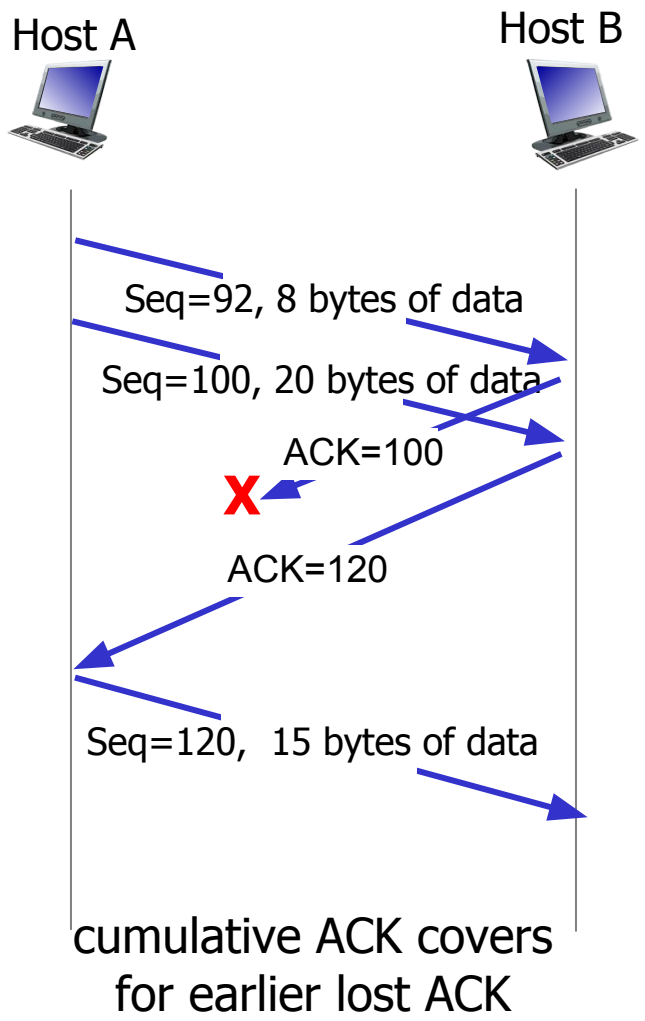


lost ACK scenario



premature timeout

TCP: retransmission scenarios



TCP fast retransmit

TCP fast retransmit

if sender receives 3 additional ACKs for same data ("triple duplicate ACKs"), resend unACKed segment with smallest seq #

- likely that unACKed segment lost, so don't wait for timeout

Receipt of three duplicate ACKs indicates 3 segments received after a missing segment – lost segment is likely. So retransmit!